15.1 Pi Agent 是什么

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~350,000 tokens，输出 ~28,000 tokens（本章合计）

在前几章，我们看了 Gateway 怎么接收消息、怎么路由到正确的会话、通道层又是如何处理消息进出的。但真正的「AI 对话」发生在哪里？答案是 **Pi Agent 运行时**。Pi Agent 是 OpenClaw 里负责跟大语言模型（LLM）打交道的核心引擎——它构建对话上下文、把请求发给模型提供者、处理流式回复、执行工具调用，最后把结果交回给 Gateway。

> **衍生解释**：Pi Agent 是 OpenClaw 的 AI 推理运行时，源自 Mario Zechner 开发的 `pi-agent-core` 库。它实现了 LLM 的 RPC 调用模式，负责管理对话上下文、工具调用和响应流式传输。可以类比为一个"AI 对话引擎"——就像浏览器中的 JavaScript 引擎负责执行代码一样，Pi Agent 负责执行 AI 对话的每一轮。

15.1.1 `@mariozechner/pi-agent-core` / `pi-ai` / `pi-coding-agent` 依赖分析

OpenClaw 的 Agent 运行时建在三个关键外部包上：

包名	角色	职责
`@mariozechner/pi-agent-core`	核心引擎	提供 LLM 对话循环、消息格式化、工具调用协议、会话持久化
`pi-ai`	AI 提供者适配	对接各个 LLM 提供者的 API（Anthropic、OpenAI、Google 等）
`pi-coding-agent`	编程能力层	提供代码执行、文件操作、终端交互等编程工具的底层实现

三个包之间是层次化的依赖关系：

```
OpenClaw 的 Agent 代码（src/agents/）
    ├── @mariozechner/pi-agent-core   ← 对话循环引擎
    │   └── pi-ai                     ← LLM API 适配层
    └── pi-coding-agent               ← 编程工具底层
```

`pi-agent-core` 定义了 Agent 运行时的核心抽象：

- **Session（会话）**——对话历史的持久化容器，存储为 JSONL 文件
- **Tool（工具）**——Agent 可以调用的外部能力，使用 JSON Schema 定义输入输出
- **Provider（提供者）**——LLM API 的统一接口，支持流式和非流式调用
- **Extension（扩展）**——在对话循环的特定阶段注入自定义逻辑的钩子

OpenClaw 并不直接用这些包的高级 API，而是通过**嵌入模式**深度集成，在 `src/agents/` 目录下搭了自己的运行时封装层。

15.1.2 RPC 模式 vs 嵌入模式

Pi Agent 有两种运行模式：

RPC 模式（外部进程）

RPC 模式下，Pi Agent 作为独立子进程运行。Gateway 通过标准输入/输出（stdin/stdout）或 HTTP 跟它通信：

```
Gateway 进程                    Pi Agent 子进程
    │                               │
    │── RPC 请求（JSON）──────────→│
    │                               │── 调用 LLM API
    │                               │← 流式回复
    │←── RPC 响应（JSON）──────────│
```

这种模式的好处是进程隔离——Agent 崩了不会拖垮 Gateway。但跨进程通信开销不小，也没法直接共享内存里的状态。

嵌入模式（同进程）

嵌入模式下，Pi Agent 的代码直接跑在 Gateway 进程里。这是 OpenClaw 当前的默认模式：

```
Gateway 进程
    │
    ├── WebSocket 服务器
    ├── HTTP 服务器
    ├── 通道适配器
    └── Pi Agent 运行时（嵌入）
        ├── 对话循环
        ├── 工具执行
        └── LLM API 调用
```

嵌入模式的好处：

- **零序列化开销**——函数调用代替 JSON-RPC
- **共享状态**——可以直接访问 Gateway 的配置、会话存储和事件系统
- **更低延迟**——无需进程间通信
- **简化部署**——单进程、单二进制

在 OpenClaw 源码里， `isCliProvider` 函数用来判断是否需要以独立 CLI 进程（RPC 模式）运行：

```
// src/agents/model-selection.ts
export function isCliProvider(provider: string, cfg?: OpenClawConfig): boolean {
  // 判断提供者是否需要 CLI 运行模式（如 claude-cli 或 codex）
  const normalized = normalizeProviderId(provider);
  return normalized === "claude-cli" || normalized === "codex" || ...;
}
```

大多数提供者（Anthropic、OpenAI、Google 等）走嵌入模式，只有少数特殊提供者（如 Claude CLI、Codex）才用 RPC 模式。

15.1.3 Agent 入口文件层次

OpenClaw 的 Agent 入口文件是**层层剥洋葱**的结构，从高级到低级依次是：

```
src/agents/pi-embedded.ts            ← 公共 API 导出（类型 + 函数重导出）
    └── pi-embedded-runner.ts        ← 模块聚合层（重导出所有子模块）
        ├── run.ts                   ← 核心运行函数 runEmbeddedPiAgent()
        ├── runs.ts                  ← 活跃运行管理（注册/查询/中止）
        ├── lanes.ts                 ← 车道（Lane）解析
        ├── model.ts                 ← 模型解析
        ├── compact.ts               ← 会话压缩
        ├── system-prompt.ts         ← 系统提示词覆盖
        ├── tool-split.ts            ← 工具分割
        └── types.ts                 ← 类型定义
```

第一层： `pi-embedded.ts` ——纯粹的重导出文件，提供稳定的公共 API：

```typescript
// src/agents/pi-embedded.ts
export type {
  EmbeddedPiAgentMeta,
  EmbeddedPiCompactResult,
  EmbeddedPiRunMeta,
  EmbeddedPiRunResult,
} from "./pi-embedded-runner.js";
export {
  abortEmbeddedPiRun,
  compactEmbeddedPiSession,
  isEmbeddedPiRunActive,
  isEmbeddedPiRunStreaming,
  queueEmbeddedPiMessage,
  resolveEmbeddedSessionLane,
  runEmbeddedPiAgent,
  waitForEmbeddedPiRunEnd,
} from "./pi-embedded-runner.js";
```

第二层： `pi-embedded-runner.ts` ——聚合层，把分散在子目录里的实现统一导出：

```
// src/agents/pi-embedded-runner.ts
export { runEmbeddedPiAgent } from "./pi-embedded-runner/run.js";
export {
  abortEmbeddedPiRun,
  isEmbeddedPiRunActive,
  isEmbeddedPiRunStreaming,
  queueEmbeddedPiMessage,
  waitForEmbeddedPiRunEnd,
} from "./pi-embedded-runner/runs.js";
export { resolveEmbeddedSessionLane } from "./pi-embedded-runner/lanes.js";
// ...
```

第三层： `pi-embedded-runner/` 子目录——实际实现所在。其中最核心的是 `run.ts` ，包含了 `runEmbeddedPiAgent` 函数——整个 Agent 循环的入口点。

核心类型定义

Agent 运行的输入输出由这几个类型定义：

```typescript
// src/agents/pi-embedded-runner/types.ts
export type EmbeddedPiRunResult = {
  payloads?: Array<{
    text?: string;          // 回复文本
    mediaUrl?: string;      // 媒体 URL
    isError?: boolean;      // 是否为错误消息
  }>;
  meta: EmbeddedPiRunMeta; // 运行元数据
  didSendViaMessagingTool?: boolean; // 是否通过消息工具直接发送
};

export type EmbeddedPiRunMeta = {
  durationMs: number;           // 运行时长（毫秒）
  agentMeta?: EmbeddedPiAgentMeta; // Agent 元数据
  aborted?: boolean;            // 是否被中止
  systemPromptReport?: SessionSystemPromptReport; // 系统提示词报告
  error?: {                     // 错误信息
    kind: "context_overflow" | "compaction_failure" | "role_ordering" | "image_size";
    message: string;
  };
};

export type EmbeddedPiAgentMeta = {
  sessionId: string;    // 会话 ID
  provider: string;     // 提供者（如 "anthropic"）
  model: string;        // 模型 ID（如 "claude-sonnet-4-20250514"）
  usage?: {             // Token 用量统计
    input?: number;
    output?: number;
    cacheRead?: number;
    cacheWrite?: number;
    total?: number;
  };
};
```

这些类型把 Agent 运行结果建模得很完整——不只是回复文本，还有执行耗时、Token 用量、是否被中止、是否出错等元信息，为上层的用量统计和错误恢复提供了必要的数据。

本节小结

1. **Pi Agent** 是 OpenClaw 的 AI 对话引擎，构建在 `pi-agent-core` 、 `pi-ai` 、 `pi-coding-agent` 三个外部库之上。
2. Agent 运行支持 **RPC 模式**（独立子进程）和**嵌入模式**（同进程），OpenClaw 默认使用嵌入模式以获得零序列化开销和共享状态的优势。
3. Agent 入口代码采用**三层结构**：公共 API 导出层 → 模块聚合层 → 子目录实现层，核心函数是 `runEmbeddedPiAgent()` 。
4. `EmbeddedPiRunResult` 类型完整建模了 Agent 运行的所有输出，包括回复内容、元数据、错误信息和用量统计。